Class 04 - Functions: from algorithm to reusable code

Week 01 - Logic | First exercise in class_04_functions

Current path: Python_Engineering / 01_python_bootcamp / week_01_logic / class_04_functions

Where we are

The previous three classes are now closed: equation -> algorithm, conditionals, and loops. We remain inside week_01_logic, but move to a new class: class_04_functions. The goal is no longer to learn syntax in isolation, but to start transforming linear scripts into reusable building blocks.

Block	Status
Class 01 - Equation to Algorithm	Completed
Class 02 - Conditionals	Completed
Class 03 - Loops	Completed
Class 04 - Functions	NOW

Class objectives

By the end of this class you should be able to explain and correctly use: def, parameters, arguments, return, local scope, function calls, and code reuse. We will also begin separating calculation, validation, and presentation.

Key idea

A function is a small machine: it receives data, performs one clear task, and, when appropriate, returns a result.

```
INPUTS  ->  FUNCTION  ->  OUTPUT
              |
              +-- internal logic
```

1. What is a function?

Until now we have written instructions one after another. If we wanted to calculate dynamic pressure ten times, we would repeat code. A function lets us encapsulate that calculation once and reuse it.

```
def calculate_dynamic_pressure(rho, speed):
    dynamic_pressure = 0.5 * rho * speed**2
    return dynamic_pressure
```

def creates the function. rho and speed are parameters. return sends the result back to the place where the function was called.

Parameters vs arguments

Concept	Example	Meaning
Parameter	rho, speed	Names used when defining the function
Argument	1.225, 100.0	Actual values used when calling it

```
q = calculate_dynamic_pressure(1.225, 100.0)
```

Here, 1.225 and 100.0 are arguments. Inside the function, Python assigns them to rho and speed respectively.

2. return is not print

This distinction is fundamental. print() displays information to the user. return provides a value that the rest of the program can continue using.

```
def bad_example(rho, speed):
    q = 0.5 * rho * speed**2
    print(q)

def useful_example(rho, speed):
    q = 0.5 * rho * speed**2
    return q
```

With the second version you can later store q, compare it, write it to a file, or use it in another equation.

3. Scope: local variables

A variable created inside a function is normally local: it exists during that call and does not necessarily exist outside the function.

```
def calculate_dynamic_pressure(rho, speed):
    q = 0.5 * rho * speed**2
    return q

result = calculate_dynamic_pressure(1.225, 100.0)
# q is local; result exists outside
```

This separation reduces errors and helps each function keep one clear responsibility.

4. Exercise 09 - Dynamic Pressure Function

File: exercises/ex09_dynamic_pressure_function.py

We will reuse the first exercise of the course, but now convert it into a function.

Physical model

q = 0.5 * rho * V^2

Exercise requirements

- Create a function named calculate_dynamic_pressure.
- The function must receive rho and speed as parameters.
- Validate rho >= 0 and speed >= 0.
- Calculate dynamic pressure.
- Return the result using return.
- The final print should be outside the function.
- Test the same function with several input pairs without copying the equation.

Important: do not use classes, modules, or input() yet. The focus is understanding functions.

5. Design before coding

```
FUNCTION
--------
Name: calculate_dynamic_pressure

INPUTS
------
rho
speed

VALIDATION
----------
rho >= 0
speed >= 0

CALCULATION
-----------
q = 0.5 * rho * speed**2

OUTPUT
------
return q
```

Do not copy a full solution. Use this design to write it yourself. The goal is to explain what goes in, what happens inside, and what comes out.

6. Test cases

rho [kg/m3]	speed [m/s]	Expected
1.225	100	6125 Pa
1.225	0	0 Pa
0	100	0 Pa
-1	100	Reject
1.225	-20	Reject

Also verify that you can call the function more than once:

```
q1 = calculate_dynamic_pressure(...)
q2 = calculate_dynamic_pressure(...)
q3 = calculate_dynamic_pressure(...)
```

If changing the case only requires changing the arguments, you have achieved your first real form of code reuse.

7. What to watch in the debugger

Place a breakpoint on the first line inside the function. Watch Python enter it, create local parameters, calculate the result, execute return, and jump back to the caller.

```
caller -> function -> return -> caller
```

8. Optional bonus

If you finish quickly, try creating a second function only for input validation. This is optional; it introduces the idea of single responsibility: one function, one clear task.

Do not move on to nested functions, lambdas, or decorators yet. First we want the foundations to be solid.

Learning Tracker

Concept	Status after EX09
def	Introduced
Parameters	Introduced
Arguments	Introduced
return	Introduced
print vs return	Introduced
Local scope	Introduced
Reuse	Initial practice
Validation inside functions	Initial practice

Next planned step

After EX09 we will remain inside week_01_logic -> class_04_functions. The next exercise will reuse the Rocket Equation to practice multiple parameters, validation, and return values. I will explicitly tell you whenever we move to a new class or a new week.